Faculdade Unicesumar

Engenharia de software

Paradigmas de Linguagem de programação

Integrantes:

Bruno Laroca,

Zoltan Goezce,

Jefferson Namur,

Severo Junior,

Luiz Bolzani

Turma: ESOFT6SNA

FindU-SISTEMA DE ACHADOS E PERDIDOS

PROJETO INTEGRADOR- PARADIGMAS EM AÇÃO

Ponta Grossa-Pr

9 de outubro de 2026

SUMÁRIO

INTRODUÇÃO....4
1 DEFINIÇÃO DO PROBLEMA E REQUISITOS....5
1.1 Definição do problema....5
1.2 Requisitos funcionais....6
1.3 Aplicação dos paradigmas de programação....7
2 MODELAGEM ORIENTADA A OBJETOS....8
2.1 Identificação das classes....9
2.2 Atributos e métodos das classes....10
2.3 Conceitos de orientação a objetos....11
2.4 Modelagem do diagrama de classes....12
3 PROTÓTIPO ORIENTADO A OBJETOS....13
3.1 Tecnologias utilizadas....13
3.2 Funcionamento proposto do protótipo....13
3.3 Implementação e testes....14
3.4 Aplicação dos conceitos de orientação a objetos....20
4 PROGRAMAÇÃO FUNCIONAL....22
4.1 Aplicação do paradigma funcional....22
4.2 Implementação da programação funcional....22
4.3 Teste de funcionalidade....23
5 PROGRAMAÇÃO LÓGICA....24
5.1 Aplicação do paradigma lógico....24
5.2 Fatos e regras....25
5.3 Consultas e resultados....26

6 PROGRAMAÇÃO ORIENTADA A EVENTOS 28
6.1 Aplicação do paradigma 28
6.2 Demonstração prática 28
7 COMPARAÇÃO ENTRE PARADIGMAS 29
7.1 Análise comparativa 29
7.2 Considerações finais 31
REFERÊNCIAS 32

Introdução

Este documento apresenta o desenvolvimento do FindU, um sistema de achados e perdidos voltado para facilitar o registro e a localização de objetos perdidos ou encontrados em ambientes universitários.

A proposta surgiu da necessidade de oferecer uma maneira mais organizada de conectar pessoas que perderam objetos com aquelas que os encontraram. Por meio do sistema, os usuários poderão cadastrar objetos, consultar registros e identificar possíveis correspondências, facilitando o processo de recuperação.

O projeto tem como objetivo aplicar diferentes paradigmas de programação na resolução de um mesmo problema. Para isso, será utilizada a linguagem Java, com interface gráfica desenvolvida em Swing, demonstrando conceitos de programação orientada a objetos, funcional e orientada a eventos. Além disso, a linguagem Prolog será utilizada para representar fatos, regras e consultas relacionadas aos objetos cadastrados. Dessa forma, o FindU busca demonstrar como diferentes paradigmas podem contribuir para o desenvolvimento de uma solução, permitindo analisar suas características, vantagens e limitações.

Etapa 1 - Definição do problema e requisitos

1.1 Definição do problema

Em ambientes universitários, é comum que alunos, professores e funcionários percam objetos pessoais, como documentos, chaves, carteiras e materiais escolares. Muitas vezes, esses objetos são encontrados por outras pessoas, mas nem sempre existe uma maneira organizada de informar onde estão ou como recuperá-los.

Atualmente, algumas dessas situações são divulgadas pelo Spotted, um perfil do Instagram utilizado pela comunidade universitária. Porém, as publicações nem sempre informam onde os objetos encontrados estão disponíveis, o que pode dificultar o contato entre quem perdeu e quem encontrou.

Pensando nessa situação, o FindU foi proposto como um sistema de achados e perdidos, permitindo que os usuários cadastrem objetos perdidos ou encontrados, informem o local relacionado ao registro e consultem possíveis correspondências. A ideia é reunir essas informações em um único ambiente, facilitando a busca e a recuperação dos objetos. O sistema será voltado principalmente para a comunidade universitária, oferecendo funcionalidades simples de cadastro, pesquisa e identificação de possíveis correspondências entre os registros.

1.2 Requisitos Funcionais

Os requisitos funcionais representam as principais funcionalidades que o FindU deverá oferecer aos usuários. Essas funcionalidades foram definidas considerando as necessidades de quem perde ou encontra objetos no ambiente universitário.

Tabela 1 – Requisitos funcionais do FindU

CÓDIGO	DESCRIÇÃO DO REQUISITO
RF01	O sistema deve permitir o cadastro de usuários.
RF02	O sistema deve permitir o registro de objetos perdidos, informando suas características e o último local conhecido.
RF03	O sistema deve permitir o registro de objetos encontrados, informando suas características e o local onde estão disponíveis para retirada.
RF04	O sistema deve permitir a consulta dos objetos cadastrados.
RF05	O sistema deve permitir a pesquisa de objetos por nome ou categoria.
RF06	O sistema deve identificar possíveis correspondências entre objetos perdidos e encontrados.
RF07	O sistema deve emitir notificações quando identificar possíveis correspondências.
RF08	O sistema deve permitir registrar a devolução de um objeto ao proprietário.

Fonte: Elaboração da equipe (2026).

1.3 Aplicação dos paradigmas de programação

O FindU permite a utilização de diferentes paradigmas de programação, pois suas funcionalidades envolvem o cadastro de objetos, a organização das informações e a identificação de possíveis correspondências entre os registros.

A programação orientada a objetos será utilizada para organizar o sistema por meio de classes, representando usuários, objetos perdidos e encontrados. Já a programação funcional será aplicada na pesquisa e filtragem dos objetos cadastrados, facilitando a consulta das informações.

A programação lógica será demonstrada por meio da linguagem Prolog, utilizando fatos e regras para identificar possíveis correspondências entre objetos. Além disso, a programação orientada a eventos será aplicada na interface gráfica desenvolvida em Java Swing, permitindo que o sistema responda às ações dos usuários, como o cadastro de objetos e o acionamento de botões. Dessa forma, será possível observar como diferentes paradigmas podem ser utilizados em um mesmo projeto, considerando suas características, vantagens e limitações.

Etapa 2 – Modelagem orientada a objetos

A programação orientada a objetos será utilizada no desenvolvimento do FindU para organizar as informações e funcionalidades do sistema por meio de classes e objetos. Essa abordagem permite representar elementos como usuários, objetos perdidos, objetos encontrados e registros de forma organizada.

Cada classe terá seus próprios atributos e métodos, responsáveis por armazenar informações e executar determinadas ações. Essa organização facilita a manutenção do código e permite que novas funcionalidades sejam adicionadas futuramente.

Para o desenvolvimento do projeto, será utilizada a linguagem Java, aplicando conceitos como abstração, encapsulamento, herança e polimorfismo, conforme as necessidades do sistema.

2.1 Identificação das classes

Para representar as principais funcionalidades do FindU, foram definidas sete classes, cada uma com uma responsabilidade específica dentro do sistema. Essas classes permitem organizar os cadastros, controlar os registros e identificar possíveis correspondências entre objetos perdidos e encontrados.

CLASSE	RESPONSABILIDADE
Usuário	Armazenar os dados dos usuários do sistema.
Objeto	Representar as características comuns dos objetos cadastrados.
ObjetoPerdido	Representar os objetos que foram perdidos.
ObjetoEncontrado	Representar os objetos encontrados.
Registro	Armazenar informações sobre cada ocorrência.
GerenciadorFindU	Gerenciar cadastros, consultas e pesquisas.
Notificação	Informar aos usuários quando houver possíveis correspondências entre objetos perdidos e encontrados.

Fonte: Elaboração da equipe (2026).

2.2 Atributos e métodos das classes

As classes do FindU possuem atributos e métodos que definem suas características e funcionalidades. Os atributos armazenam informações importantes para o sistema, enquanto os métodos permitem realizar ações relacionadas aos usuários, objetos e registros.

A tabela a seguir apresenta os principais atributos e métodos definidos para cada classe.

CLASSE	**ATRIBUTOS**	**MÉTODOS**
Usuário	id, nome, email	getNome(), getEmail()
Objeto	id, nome, categoria, descricao	getNome(), getCategoria(), getTipoRegistro()
ObjetoPerdido	ultimoLocalConhecido	getTipoRegistro()
ObjetoEncontrado	localEncontrado, localRetirada	getTipoRegistro()
Registro	id, usuario, objeto, data, status	atualizarStatus(), getObjeto()
GerenciadorFindU	usuarios, registros	cadastrarObjeto(), buscarObjetos(), buscarCorrespondencias()
Notificacao	mensagem, destinatario	exibirMensagem()

Fonte: Elaboração da equipe (2026).

2.3 Conceitos de orientação a objetos

No desenvolvimento do FindU, serão utilizados os principais conceitos da programação orientada a objetos, como abstração, encapsulamento, herança e polimorfismo. Esses conceitos ajudam a organizar as classes e definir como os elementos do sistema se relacionam.

Abstração: será aplicada na classe Objeto, que reúne características comuns aos objetos cadastrados, como nome, categoria e descrição.

Encapsulamento: os atributos das classes serão definidos como privados, permitindo que suas informações sejam acessadas ou alteradas por meio de métodos específicos.

Herança: as classes ObjetoPerdido e ObjetoEncontrado herdarão as características da classe Objeto, evitando a repetição de atributos e métodos.

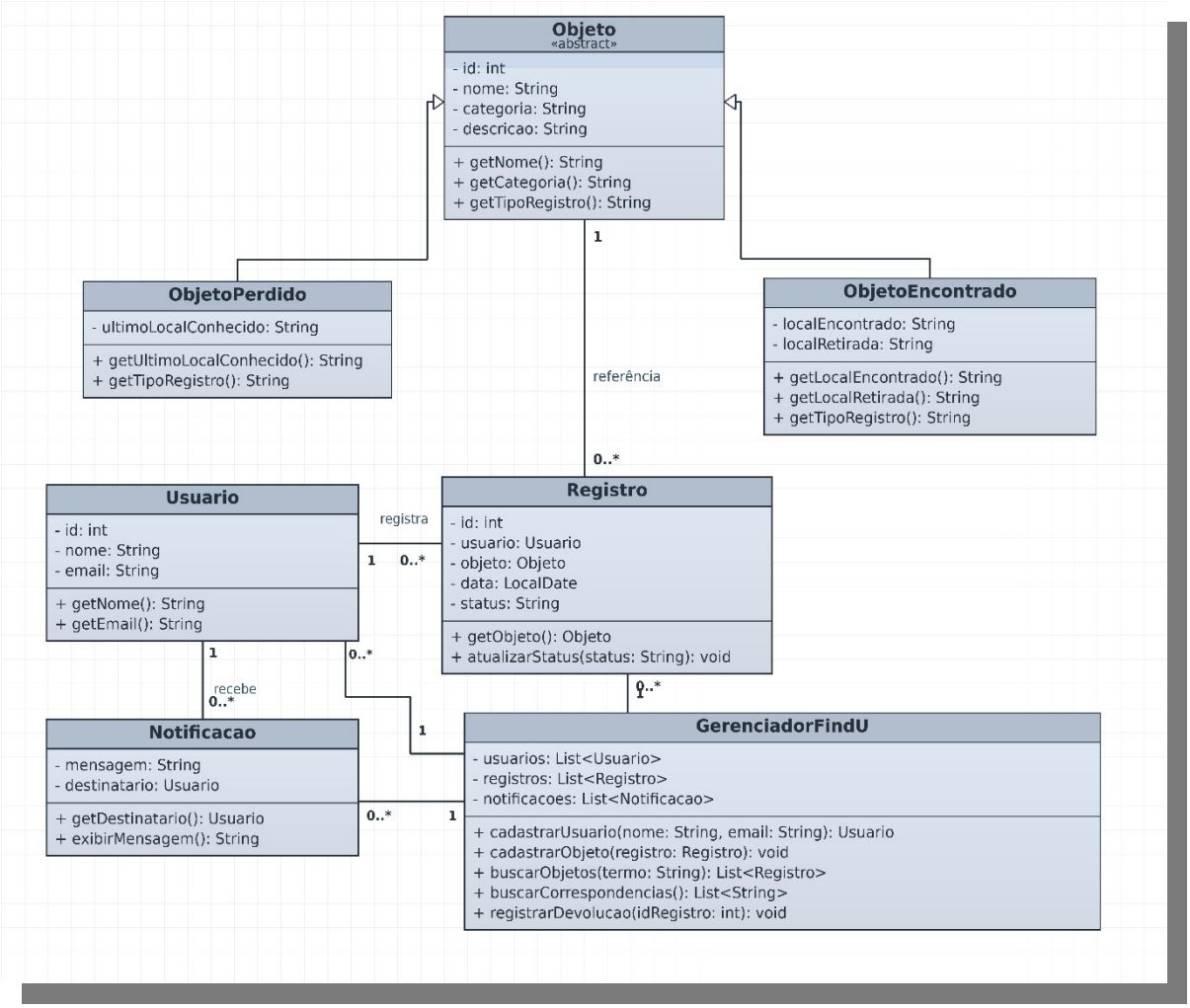
Polimorfismo: será demonstrado pelo método getTipoRegistro(), que terá comportamentos diferentes nas classes ObjetoPerdido e ObjetoEncontrado, identificando o tipo de cada objeto.

Dessa forma, esses conceitos serão aplicados na organização do código Java, facilitando a compreensão e a manutenção do sistema.

2.4 Modelagem do diagrama de classes

O diagrama de classes do FindU apresenta a estrutura do sistema e os relacionamentos entre suas principais classes. Nele, é possível visualizar os atributos, métodos e a relação de herança entre as classes Objeto, ObjetoPerdido e ObjetoEncontrado. Essa representação facilita a compreensão de como o sistema será organizado e servirá como base para o desenvolvimento do protótipo em Java.

Figura 1 - Diagrama de classes do FindU



Fonte: Elaboração da equipe (2026).

Etapa 3 - Protótipo Orientado a Objetos

Nesta etapa, será desenvolvido um protótipo funcional do FindU, utilizando a programação orientada a objetos para colocar em prática as classes e os conceitos apresentados anteriormente.

O protótipo terá como objetivo demonstrar algumas das principais funcionalidades do sistema, como o cadastro de objetos perdidos e encontrados, a consulta dos registros e a identificação de possíveis correspondências.

A implementação será realizada de forma local, permitindo verificar o funcionamento das classes e a interação entre os diferentes elementos do sistema.

3.1 Tecnologias utilizadas

Para o desenvolvimento do protótipo, será utilizada a linguagem Java, por oferecer suporte aos principais conceitos da programação orientada a objetos, como encapsulamento, herança e polimorfismo.

A interface gráfica será desenvolvida com a biblioteca Java Swing, permitindo criar janelas, botões, campos de preenchimento e tabelas para interação com os usuários.

O desenvolvimento será realizado no Visual Studio Code (VS Code), com execução local no computador. Nesta primeira versão, os dados serão armazenados temporariamente em memória, sem a necessidade de utilizar um banco de dados.

3.2 Funcionamento proposto do protótipo

O protótipo do FindU permitirá cadastrar objetos perdidos e encontrados por meio de uma interface gráfica simples. Durante o cadastro, o usuário poderá informar características do objeto, como nome, categoria, descrição e local relacionado à ocorrência.

Após o cadastro, os registros poderão ser consultados e pesquisados. O sistema também terá uma função para identificar possíveis correspondências entre objetos perdidos e encontrados, considerando as informações registradas.

Essas funcionalidades serão utilizadas para demonstrar a criação de objetos, a utilização de métodos e a aplicação dos conceitos de orientação a objetos no FindU.

3.3 Implementação e testes

Após o desenvolvimento do protótipo em Java, foram realizados testes para verificar o funcionamento das principais funcionalidades do FindU. Entre elas estão o cadastro de usuários, o registro de objetos perdidos e encontrados, a consulta dos registros e a identificação de possíveis correspondências.

Os testes foram realizados por meio da interface gráfica, utilizando informações de exemplo para simular situações que poderiam ocorrer no ambiente universitário. As figuras a seguir apresentam as telas do sistema durante sua execução.

Figura 2 - Tela inicial do sistema FindU



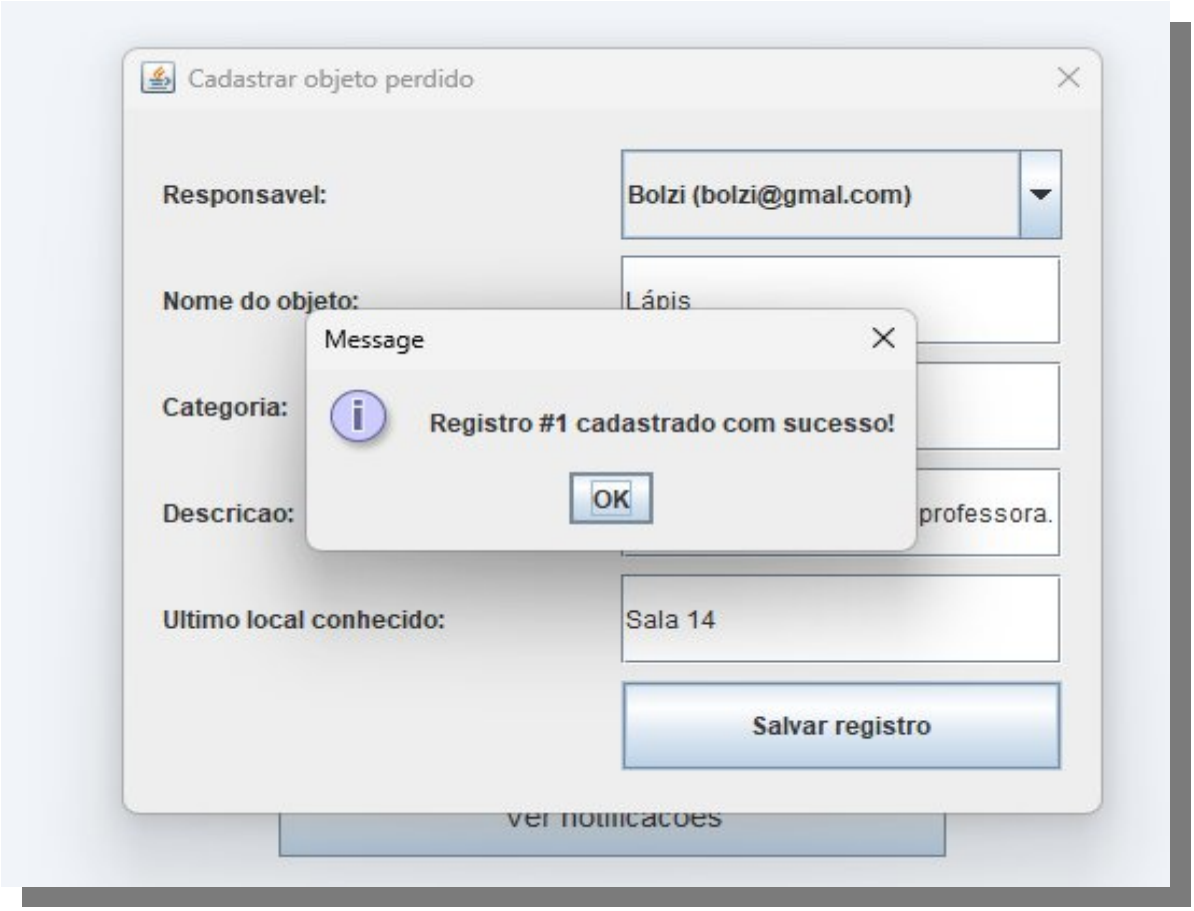
Fonte: Elaboração da equipe (2026).

Figura 3 - Confirmação do cadastro de usuário



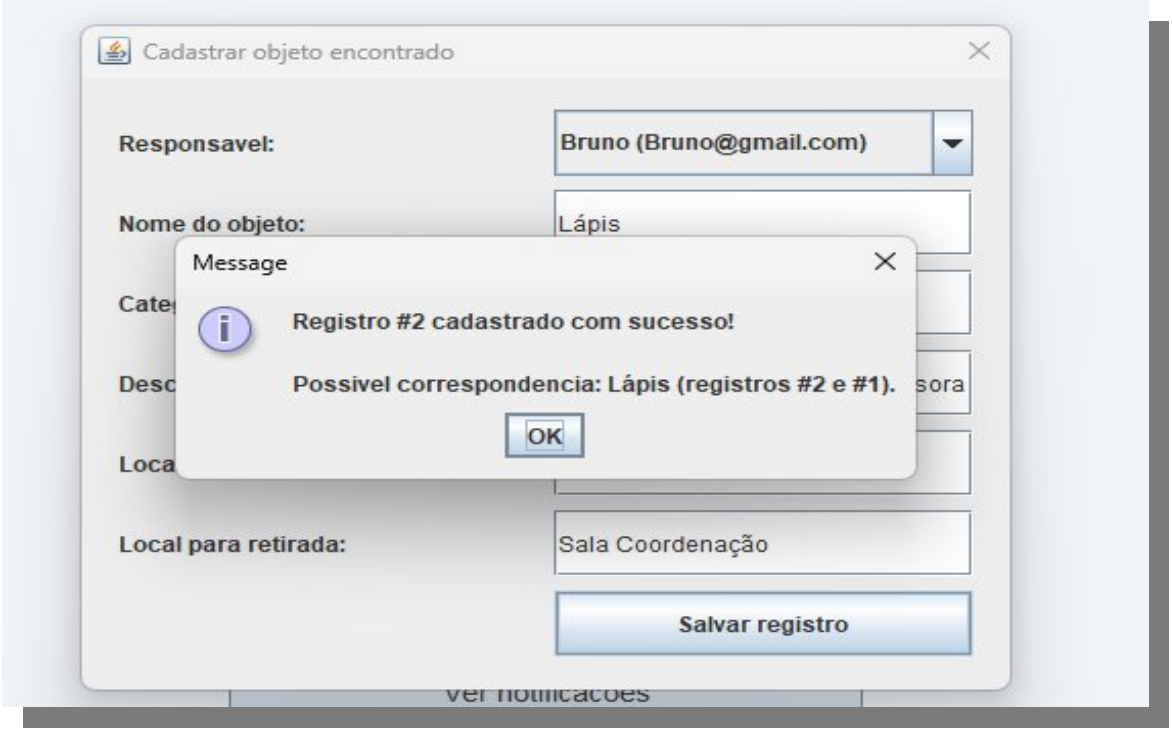
Fonte: Elaboração da equipe (2026).

Figura 4 - Confirmação do cadastro de objeto perdido



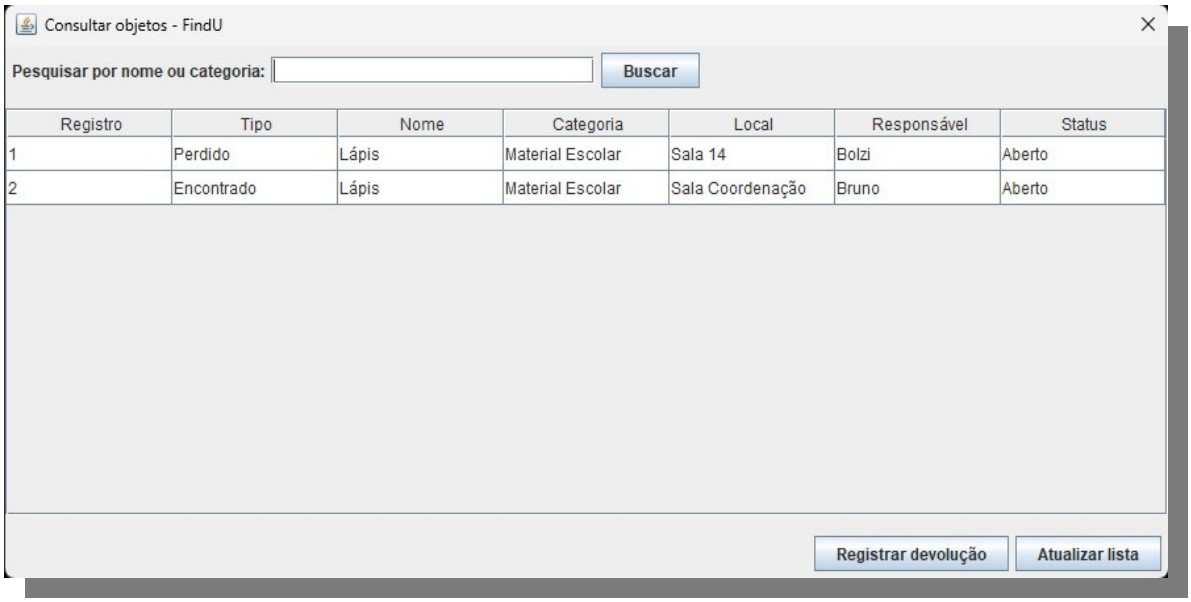
Fonte: Elaboração da equipe (2026).

Figura 5 - Confirmação do cadastro de objeto encontrado



Fonte: Elaboração da equipe (2026).

Figura 6 - Consulta dos objetos cadastrados



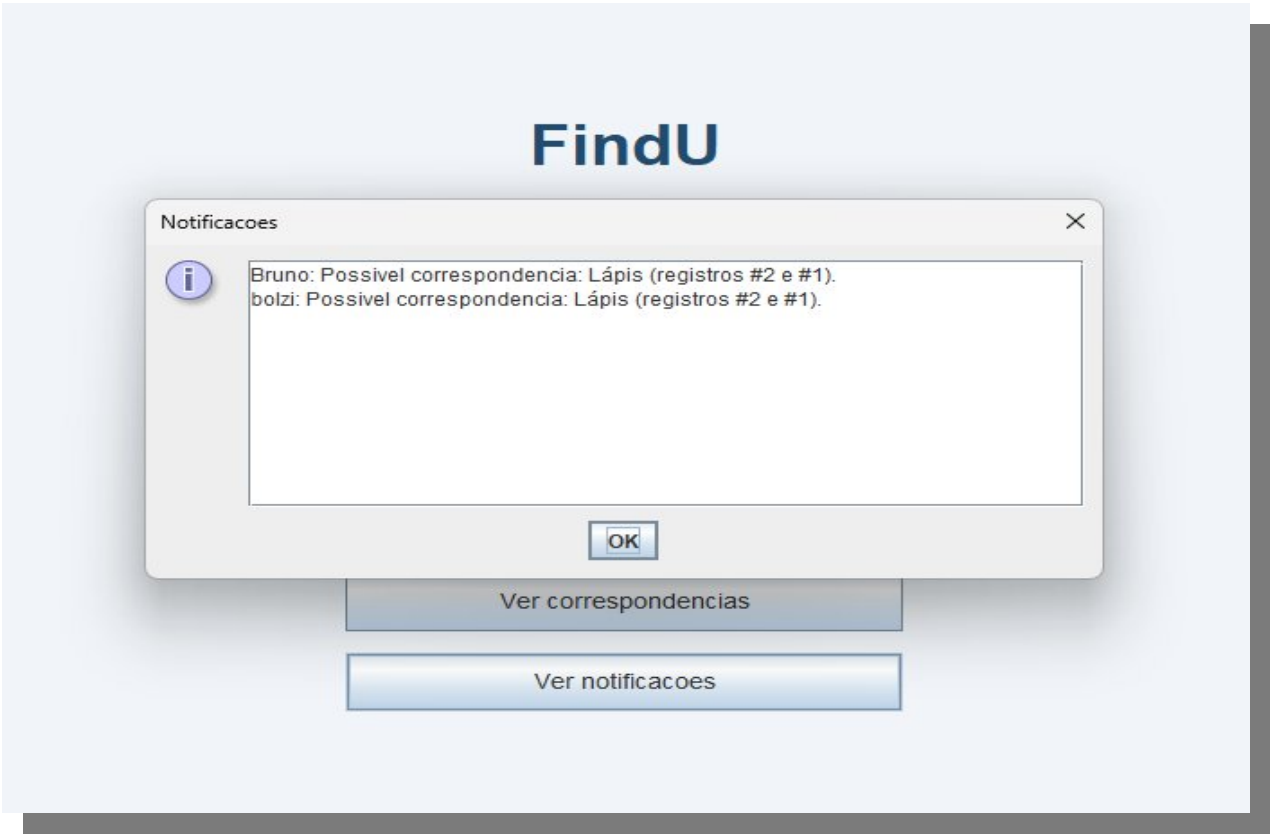
Fonte: Elaboração da equipe (2026).

Figura 7 - Identificação de possíveis correspondências



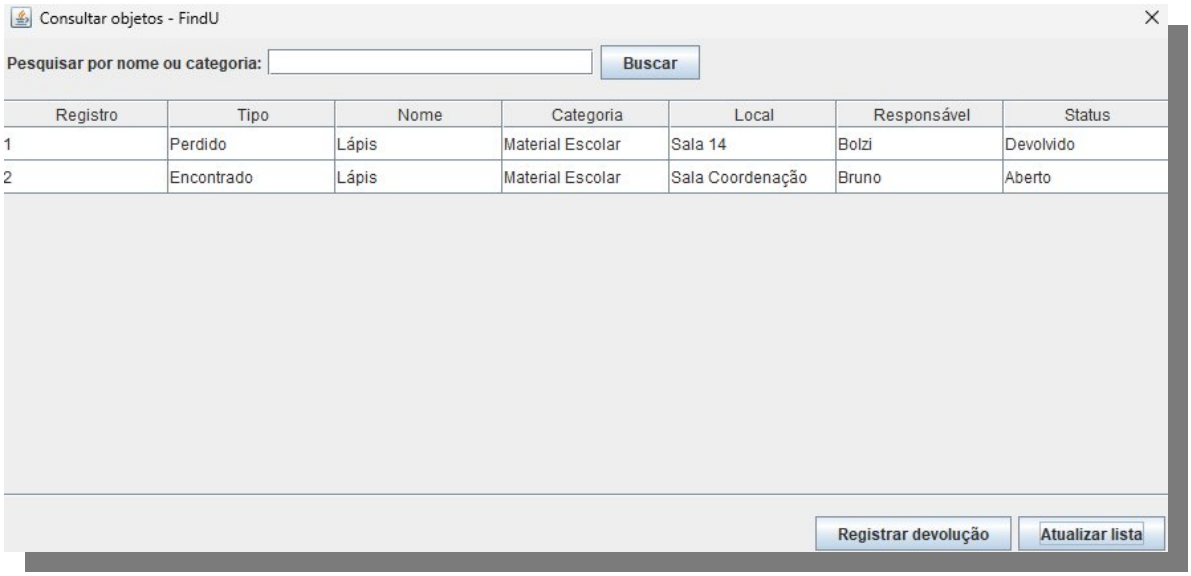
Fonte: Elaboração da equipe (2026).

Figura 8 - Visualização das notificações



Fonte: Elaboração da equipe (2026).

Figura 9 - Registro de devolução de objeto



Fonte: Elaboração da equipe (2026).

3.4 Aplicação dos conceitos de orientação a objetos

O protótipo FindU foi desenvolvido utilizando conceitos de orientação a objetos, como encapsulamento, herança, abstração e polimorfismo. Esses conceitos foram aplicados na organização das classes responsáveis pelos usuários, objetos e registros do sistema.

A classe abstrata Objeto representa as características comuns dos itens cadastrados, enquanto as classes ObjetoPerdido e ObjetoEncontrado possuem informações específicas de cada tipo de registro.

Exemplo 1 – Abstração e encapsulamento

```java
public abstract class Objeto {
    private final int id;
    private final String nome;
    private final String categoria;
    private final String descricao;
```

A classe Objeto foi definida como abstrata para representar as características comuns dos objetos cadastrados. Seus atributos são privados, garantindo o encapsulamento dos dados.

Exemplo 2 – Herança e polimorfismo

```java
public class ObjetoPerdido extends Objeto {
```

```java
    @Override
    public String getTipoRegistro() {
        return "Perdido";
    }
```

A classe ObjetoPerdido utiliza herança ao estender a classe Objeto, aproveitando suas características comuns. O polimorfismo é demonstrado pela sobrescrita do método getTipoRegistro(), que retorna o tipo específico do objeto cadastrado.

Etapa 4 - Programação Funcional

4.1 Aplicação do Paradigma Funcional

A programação funcional foi utilizada no FindU para auxiliar na busca e filtragem dos objetos cadastrados. Para isso, foram empregados recursos do Java, como Streams e funções anônimas, que permitem trabalhar com coleções de dados de forma mais simples e organizada.

Essa abordagem facilita a consulta dos registros e reduz a necessidade de utilizar estruturas repetitivas para realizar determinadas operações.

4.2 Implementação da programação funcional

Exemplo - Filtragem de objetos com Streams e funções anônimas

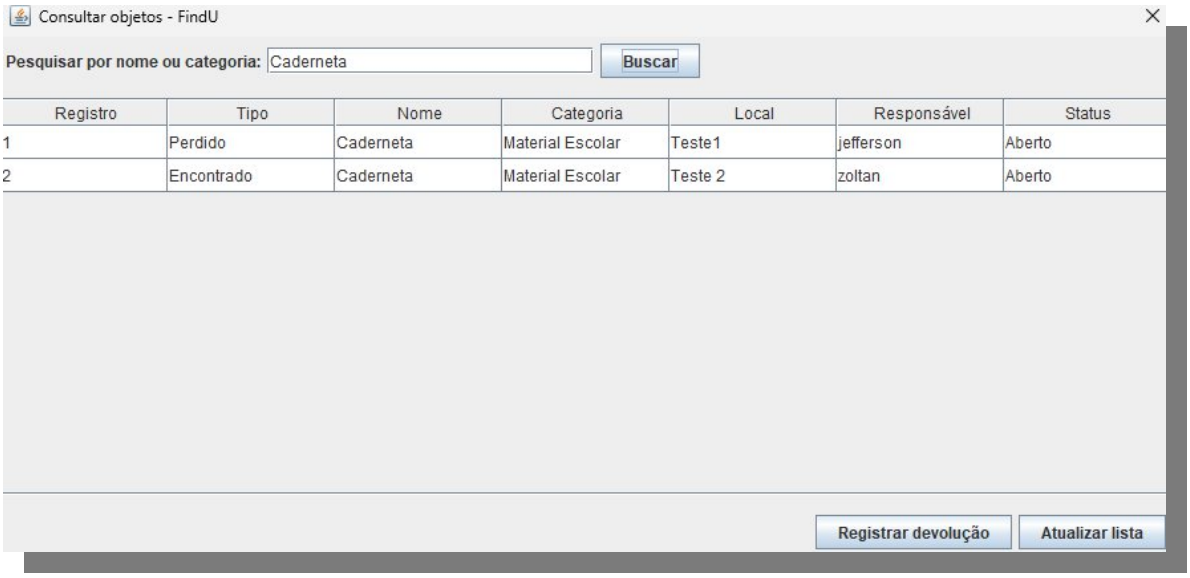
```
public List<Registro> buscarObjetos(String termo) {
    String busca = normalizar(termo == null ? "" : termo);

    return registros.stream()
        .filter(r ->
            normalizar(r.getObjeto().getNome()).contains(busca)
            || normalizar(r.getObjeto().getCategoria()).contains(busca)
        )
        .collect(Collectors.toList());
}
```

O método buscarObjetos utiliza Stream e uma função anônima para filtrar os registros cadastrados. A busca considera o nome ou a categoria do objeto, retornando uma lista com os resultados encontrados. Dessa forma, o processamento é realizado sem alterar a lista original de registros.

4.3 – Teste de Funcionalidade

Para testar a filtragem, foi realizada uma pesquisa pelo nome "Caderneta". O sistema retornou dois registros correspondentes, sendo um objeto perdido e outro encontrado. O resultado demonstra o funcionamento da busca implementada com Streams e função anônima.



Fonte: Elaboração da equipe (2026).

Na orientação a objetos, a busca faz parte do gerenciamento dos registros do sistema. Já na abordagem funcional, a filtragem é realizada por meio de funções aplicadas à lista de objetos, tornando o processamento mais direto e sem alterar os dados originais.

Etapa 5 – Programação Lógica

5.1 Aplicação do paradigma lógico

A programação lógica foi aplicada no FindU utilizando a linguagem Prolog. Foi criada uma pequena base de conhecimento com informações sobre objetos perdidos e encontrados no ambiente universitário.

O sistema utiliza fatos para representar os registros e regras para identificar possíveis correspondências. Dessa forma, é possível realizar consultas e obter resultados a partir das informações cadastradas.

5.2 Fatos e regras

A base de conhecimento do FindU contém oito fatos e duas regras. Os fatos representam objetos perdidos e encontrados, enquanto as regras permitem verificar possíveis correspondências e identificar objetos encontrados sem registro de perda.

Código Prolog do sistema FindU

prolog > findu.pl

```
% FindU - Sistema de Achados e Perdidos

% Objetos perdidos
perdido(carteira, acessorios, biblioteca).
perdido(chave, acessorios, laboratorio).
perdido(caderno, material_escolar, sala_101).
perdido(celular, eletronicos, cantina).

% Objetos encontrados
encontrado(carteira, acessorios, biblioteca).
encontrado(chave, acessorios, laboratorio).
encontrado(mochila, acessorios, recepcao).
encontrado(celular, eletronicos, cantina).

% Regra 1 - Possiveis correspondencias
possivel_correspondencia(Objeto) :-
    perdido(Objeto, Categoria, _),
    encontrado(Objeto, Categoria, _).

% Regra 2 - Encontrados sem registro de perda
sem_registro_de_perda(Objeto) :-
    encontrado(Objeto, _, _),
    \+ perdido(Objeto, _, _).
```

Fonte: Elaboração da equipe (2026).

5.3 Consultas e resultados

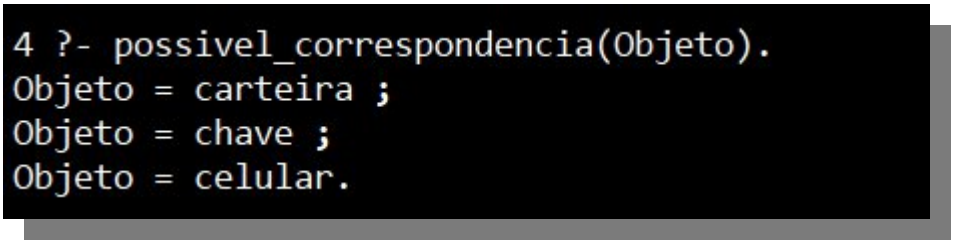
Foram realizadas quatro consultas no SWI-Prolog para verificar o funcionamento da base de conhecimento. Os testes permitiram listar objetos perdidos e encontrados, identificar possíveis correspondências e localizar objetos encontrados sem registro de perda.

CONSULTA REALIZADA	**RESULTADO OBTIDO**
perdido(Objeto, Categoria, Local).	Carteira, chave, caderno e celular.
encontrado(Objeto, Categoria, Local).	Carteira, chave, mochila e celular.
possivel_correspondencia(Objeto).	Carteira, chave e celular
sem_registro_de_perda(Objeto).	Mochila

Fonte: Elaboração da equipe (2026).

Os resultados mostram que a regra de possíveis correspondências identifica objetos que possuem o mesmo nome e categoria nos registros de perdidos e encontrados. Por isso, carteira, chave e celular foram identificados como possíveis correspondências. Já a mochila foi encontrada, mas não possui um registro de perda correspondente, sendo identificada pela segunda regra.

Evidência 1. Identificação de possíveis correspondências no Prolog



Fonte: Elaboração da equipe (2026).

Evidência 2. Objeto sem registro de perda

```
5 ?- sem_registro_de_perda(Objeto).
Objeto = mochila .
```

Fonte: Elaboração da equipe (2026).

Etapa 6 - Programação Orientada a Eventos

6.1 Aplicação do Paradigma

A programação orientada a eventos foi utilizada no FindU por meio da interface gráfica desenvolvida com Java Swing. Nesse modelo, determinadas ações do sistema são executadas a partir da interação do usuário, como clicar em botões ou realizar uma pesquisa.

No FindU, os botões permitem acessar as funcionalidades de cadastro, consulta, correspondências e notificações. Dessa forma, o sistema responde aos eventos realizados pelo usuário, facilitando a utilização da aplicação.

6.2 Demonstração prática

No FindU, a programação orientada a eventos é demonstrada pelo uso do método addActionListener(), responsável por identificar o clique do usuário em um botão e executar a ação correspondente.

Esse recurso permite que as funcionalidades do sistema sejam acionadas diretamente pela interface gráfica, sem a necessidade de executar comandos manualmente.

Demonstração prática:

```
JButton botao = new JButton(texto);
botao.setPreferredSize(new Dimension(width: 310, height: 39));
botao.setFont(new Font(name: "Arial", Font.PLAIN, size: 14));
botao.addActionListener(e -> acao.run());
```

Fonte: Elaboração da equipe (2026).

Etapa 7 - Comparação entre paradigmas

7.1 Análise comparativa

Durante o desenvolvimento do FindU, foram utilizadas diferentes abordagens de programação para representar partes do mesmo problema. Cada uma apresentou características próprias, contribuindo para a organização, o funcionamento e a consulta das informações do sistema.

Quadro – Comparação dos paradigmas utilizados no FindU

CRITÉRIO	ORIENTAÇÃO A OBJETOS	FUNCIONAL	LÓGICA	ORIENTADA A EVENTOS
Foco	**Representar entidades do sistema**	**Processar dados**	**Consultar fatos e regras**	**Responder às ações do usuário**
Organização	**Classes e métodos**	**Funções e operações**	**Fatos, regras e consultas**	**Eventos e ações**
Estado e dados	**Objetos armazenam informações**	**Filtragem sem alterar a lista original**	**Base de conhecimento**	**Ações podem alterar o estado**
Reutilização	**Classes e métodos**	**Funções**	**Regras**	**Tratadores de eventos**
Legibilidade	**Facilidade de entender a estrutura**	**Busca mais direta**	**Regras de fácil interpretação**	**Liga ações á interface**
Adequação ao FindU	**Cadastro e gerenciamento**	**Pesquisa de objetos**	**Identificação de correspondências**	**Funcionamento dos botões**
Vantagens	**Organização e manutenção**	**Código mais simples**	**Consultas declarativas**	**Interface interativa**
Limitações	**Exige uma maior estruturação**	**Não substitui toda a modelagem**	**Depende de fatos cadastrados**	**Precisa das demais funcionalidades**

Fonte: Elaboração da equipe (2026).

Além do FindU, os paradigmas estudados também podem ser utilizados no mercado de trabalho. A programação orientada a objetos é bastante utilizada no desenvolvimento de sistemas e aplicativos. A programação funcional ajuda no processamento e na filtragem de dados, enquanto a programação lógica pode ser aplicada em sistemas baseados em regras. Já a programação orientada a eventos é comum em interfaces gráficas, pois permite que os programas respondam às ações dos usuários.

7.2 Considerações finais

Não existe um paradigma de programação que seja melhor para todos os problemas, pois cada abordagem possui características, vantagens e limitações diferentes.

No desenvolvimento do FindU, a orientação a objetos foi importante para organizar as classes e os registros. A programação funcional facilitou a busca de objetos, enquanto a programação lógica permitiu identificar possíveis correspondências por meio de fatos e regras. Já a programação orientada a eventos tornou possível a interação do usuário com a interface gráfica.

Dessa forma, o projeto demonstrou que diferentes paradigmas podem ser utilizados em conjunto, de acordo com as necessidades de cada parte do sistema.

REFERÊNCIAS

ORACLE. **Stream (Java SE 21 & JDK 21)**. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/Stream.html. Acesso em: 9 out. 2026.

ORACLE. **How to Write an Action Listener**. Disponível em: https://docs.oracle.com/javase/tutorial/uiswing/events/actionlistener.html. Acesso em: 9 out. 2026.

SWI-PROLOG. **SWI-Prolog Manual: Getting Started Quickly**. Disponível em: https://www.swi-prolog.org/man/quickstart.html. Acesso em: 9 out. 2026.